

Working Backwards in Code

Explore, Plan, Write Agents

as the Computational Press Release

Sean Chatman
ChatmanGPT

April 17, 2026

Abstract

Amazon’s Working Backwards process begins with the press release—a description of the finished product written before a line of code exists. The press release is not a lie. It is a design instrument: a future state held in present tense to pull a team toward it. This paper argues that frontier software development is structurally identical to Working Backwards, and that Claude Code’s Explore/Plan/Write agent architecture is its computational implementation. The *explore agent* measures the gap between press release and reality. The *plan agent* prioritises which part of the gap to close next. The *write agent* closes it. The one-task-at-a-time sequential cycle fails frontier work not because tasks are hard but because it destroys the press release: each completed ticket becomes its own end, severed from the original vision. We demonstrate with the `pictl/bcinr` arc, where an ambitious process-mining registry and a branchless algorithm library were each press releases written before their implementations existed—and where the agent architecture was the mechanism for measuring and closing the gap between aspiration and ground truth.

Contents

1	Is the Press Release a Lie?	2
2	Working Backwards: The Press Release as Engineering Primitive	2
2.1	The Structure of Working Backwards	2
2.2	The Computational Equivalent	3
2.3	Why Frontier Work Requires the Press Release First	3
3	The Agent Architecture as Working Backwards Engine	3
3.1	The Plan File Is the Press Release	3
3.2	Explore Agents Measure the Gap	4
3.3	Write Agents Close the Gap	4
4	Why the Sequential Task Cycle Destroys the Press Release	4
4.1	The Ticket as the Atom of Sequential Cycles	4
4.2	Ticket Completion Severs Tasks from the Press Release	5
4.3	Sequential Cycles Measure Velocity, Not Gap	5
4.4	Sequential Cycles Cannot Detect Architectural Gaps	5
5	The pictl Arc as Working Backwards in Practice	6
5.1	The Press Release: 41 Algorithms	6
5.2	The Explore Phase: Measuring the Gap	6
5.3	The Plan Revision: A Closer-Horizon Press Release	6
5.4	The bcinr Press Release Written in Code	7
6	Implications for Agent-Driven Development	7
6.1	The Plan File Must Be the Press Release, Not the Task List	7
6.2	Explore Agents Run in Parallel Because Gaps Are Structural	8
6.3	False Completion Is Press Release Revision, Not Failure	8
7	Conclusions	8

1. Is the Press Release a Lie?

Amazon's Working Backwards process requires teams to write the press release *first*—before the product spec, before the architecture review, before any code. The press release describes the product as if it already ships. It uses present tense. It quotes imaginary customers. It states capabilities that do not yet exist.

Is it wrong? Is it dishonest? Is it a lie?

No. The press release is a **design instrument**. Its purpose is not to describe what exists but to make vivid what should exist, so that everyone building toward it shares the same target. The press release is a commitment device operating on desired reality, not a report operating on current reality.

Now consider the `pictl` algorithm registry.

The registry claimed 41 algorithms: DFG, Alpha++, Inductive Miner, ILP, Genetic Algorithm, 36 more. It declared output types, quality scores, speed scores, deployment profiles. Most of these algorithms were stubs or had architectural defects that prevented them from running on real data.

Is the registry a lie?

Only if you read it as documentation. If you read it as a press release—a description of the intended final system, written in code before the implementations existed—then it is exactly correct. It is Working Backwards applied to a software registry: *this is the system we are building toward*.

The error was not writing the press release. The error was losing track of which parts were press release and which were production—and having no systematic mechanism for measuring the gap between the two.

Central claim of this paper: The purpose of the Explore/Plan/Write agent architecture is to hold the press release (plan), measure the gap between press release and reality (explore), and close it incrementally (write). The one-task-at-a-time sequential cycle fails frontier work because it orphans each task from the press release, turning means into ends.

2. Working Backwards: The Press Release as Engineering Primitive

2.1. The Structure of Working Backwards

Amazon's Working Backwards process has four artefacts, produced in this order:

1. **Press Release (pr):** The product as it will be described on launch day. Written in plain language. Includes the headline, the problem solved, the key features, and a customer quote. Present tense. Future state.
2. **FAQ:** The hard questions the press release glosses over. What happens when X fails? How does it handle Y at scale? What does it cost?
3. **User Experience Narrative:** A day-in-the-life story walking through the product from a user's perspective, in enough detail to expose design gaps.
4. **Visual / Mock:** Screenshots, diagrams, or prototypes showing what the product looks like—before building it.

Only after all four are complete does implementation begin. The press release does not change during implementation unless the product fundamentally changes. It is the fixed star that

implementation navigates by.

2.2. The Computational Equivalent

In a software system, the press release is expressed not in English but in types, APIs, registries, and configuration schemas. The `pictl` registry was a press release written in TypeScript:

```
1 {
2   id: 'inductive_miner',
3   name: 'Inductive Miner',
4   outputType: 'tree',
5   quality: { fitness: 0.90, precision: 0.85 },
6   speed: 30,
7 }
```

This says: *we are building a system that includes an inductive miner with these properties*. It is a design commitment, not an implementation claim. The FAQ for this press release would include: “What does the `WASM` function actually return?” The UX narrative would include: “A user calls `discover_inductive_miner` and receives a process tree.” The visual would be the output schema.

None of these existed. That is the gap. The gap is not evidence that the press release was wrong. It is the measurement of how much work remains.

2.3. Why Frontier Work Requires the Press Release First

Conventional software development can work bottom-up: implement a module, test it, add it to the system. This works because the problem domain is understood. The shape of the solution is known before implementation begins.

Frontier work—new algorithms, new runtime targets, new architectural patterns—cannot work bottom-up. The shape of the solution is precisely what is unknown. Bottom-up exploration produces a collection of implemented pieces with no coherent target.

The press release solves this by establishing the target *before* the shape of the solution is known. Implementation proceeds top-down from the press release, not bottom-up from components. The question is never “what should I implement next?” but “what is the largest gap between the press release and reality, and how do I close it?”

3. The Agent Architecture as Working Backwards Engine

Claude Code’s Explore/Plan/Write agent architecture maps precisely onto the Working Backwards process.

Working Backwards	Agent Type	Output
Press Release (desired state)	Plan Agent	Plan file
Gap measurement (FAQ, UX)	Explore Agent	Signal (facts about reality)
Implementation (close the gap)	Write Agent	State change (code, tests)

3.1. The Plan File Is the Press Release

The plan file—a single Markdown document updated across sessions—is the computational press release. It is:

- Written before implementation, describing the desired end state

- Revised when the product fundamentally changes (not when a task is hard)
- The fixed star that write agents navigate by
- Visible to the human before any implementation begins

When the `pict1` arc reached the conclusion that 0 of 41 algorithms were production-ready, the plan file was not amended with a note. It was **rewritten**—from “audit and fix 41 algorithms” to “implement 12 branchless families in `bcinr`”. This is exactly what Working Backwards prescribes: when the product changes, the press release changes. Everything else follows from the new press release.

3.2. Explore Agents Measure the Gap

An explore agent’s job is to produce a precise measurement of the gap $\Delta = \text{press release} - \text{reality}$.

In the `pict1` arc:

- Press release: 41 algorithms, all callable, all returning typed outputs, all measurable for fitness, precision, generalization, simplicity.
- Reality (discovered by explore agents): 0 production-ready, 24 with mutex panics, 2 returning wrong output types, 2 stubs calling DFG internally.
- Gap: $\Delta = 41$ algorithms deep, architectural root cause is `RefCell<Mutex>` in single-threaded WASM.

The gap measurement did not make the press release wrong. It made the gap *legible*. A legible gap can be evaluated: is it worth closing, or should the press release be revised to a closer horizon?

3.3. Write Agents Close the Gap

Write agents are gap-closure engines. They receive a specific gap—“the `mask` family has 3 functions that need SIMD-safe implementations”—and produce code that closes it. They do not decide what to implement. They do not evaluate whether the press release is correct. They close the gap they were given.

This division of labour is critical. An agent that simultaneously evaluates the press release and implements toward it will collapse the two roles: it will implement what it can implement rather than what the press release requires. This is how registries get populated with stubs.

4. Why the Sequential Task Cycle Destroys the Press Release

4.1. The Ticket as the Atom of Sequential Cycles

The dominant model of software development treats the *ticket* as the atomic unit of work. A ticket has a title, a description, acceptance criteria, and a status: To Do, In Progress, Done. Tickets are created, assigned, worked, and closed. Velocity is measured in tickets per sprint.

This model is excellent for known work. It is catastrophic for frontier work.

The ticket model has one implicit contract: **tickets are completable**. Every ticket has a Done state that can be reached by a competent engineer in a bounded amount of time. The system has no representation for a ticket that *should not be completed*—a ticket that is, in Working Backwards terms, a gap that the press release revision has made irrelevant.

4.2. Ticket Completion Severs Tasks from the Press Release

Consider the `pictl` inductive miner task. As a ticket, it reads:

```
1 Title: Implement discover_inductive_miner
2 Status: In Progress
3 Acceptance: Function is callable, returns output
```

A developer under time pressure closes this ticket by implementing a stub: `discover_inductive_miner` calls `discover_dfg` and returns its output. The acceptance criteria are technically met. The ticket is marked Done.

Is this wrong? In the ticket model: no. The function is callable. It returns output. Done.

In the Working Backwards model: yes. The press release says the inductive miner returns a *process tree*, not a DFG. The stub closes the ticket while widening the gap between press release and reality. The Done state of the ticket has become an obstacle to the press release.

This is the core failure mode: **ticket Done is not press release Done**. A ticket can be marked Done while the gap to the press release grows. Over enough sprints, the registry fills with Done tickets and a press release that is further from reality than when the project started.

4.3. Sequential Cycles Measure Velocity, Not Gap

Sprint velocity measures tickets closed per unit time. It is an excellent metric for known work, where each ticket is an independent unit of value.

For frontier work, velocity is worse than useless. It measures the rate at which the team is producing Done tickets—which, as shown above, can be entirely decorrelated from the rate at which the gap to the press release is closing.

A team that closes 10 stub tickets per sprint is producing high velocity and negative progress. The gap is growing. The press release is receding. But the burndown chart looks excellent.

The correct metric for frontier work is $\Delta\Delta$ per unit time: the rate of gap closure toward the press release. This metric requires knowing both the press release (the desired state) and the current reality (the measured state). The ticket model provides neither.

4.4. Sequential Cycles Cannot Detect Architectural Gaps

The deepest problem with sequential task cycles is that they explore the system one task at a time. If task i fails, the team moves to task $i + 1$. The failure of task i is treated as a local defect, not as a signal about the architecture.

In the `pictl` arc, sequential debugging would have found:

- Task 1: `alpha_plus_plus` crashes. Fix: unclear. Status: blocked.
- Task 2: `genetic_algorithm` crashes. Fix: unclear. Status: blocked.
- Task 3: `aco` crashes. Fix: unclear. Status: blocked.

Three sprints in, three blocked tickets. The architectural conclusion—that `RefCell<Mutex>` is incompatible with single-threaded WASM—is not visible. It requires seeing all three failures simultaneously and asking: what do these have in common?

Parallel exploration—launching explore agents against all 41 algorithms simultaneously—produced this architectural conclusion in a single session. The pattern was immediately legible because all the data arrived together.

Sequential cycles surface local facts. Parallel exploration surfaces architectural facts. Architectural facts are the only ones that can revise the press release.

5. The pictl Arc as Working Backwards in Practice

5.1. The Press Release: 41 Algorithms

The `pictl` registry was a press release written in TypeScript. It described a process mining platform with 41 algorithms spanning discovery, conformance, ML analysis, streaming, import/-export, and autonomic control. Each algorithm had metadata: speed, quality, output type, deployment profiles.

This was not a lie. It was a vision. The team that populated the registry was doing exactly what Working Backwards prescribes: stating the desired end state in concrete, evaluable terms before implementation.

The press release had an implicit FAQ that was never written:

- What does each WASM function actually return?
- Which algorithms have been tested against real event logs?
- Which WASM exports actually exist vs. which are planned?

These questions were not asked. The gap was not measured. Development proceeded toward individual tickets without measuring distance from the press release.

5.2. The Explore Phase: Measuring the Gap

The adversarial audit was the first systematic gap measurement. It asked, for each of the 41 algorithms:

1. Does the WASM export exist?
2. Does it execute without crashing?
3. Does its output match the declared type?
4. Does it produce a fitness score ≥ 0.85 on a real event log?
5. Does it complete in $< 5s$ on 500K events?

The audit ran on real process mining datasets: Sepsis Cases (5 MB, 476 traces, 15,000 events) as a smoke test, and Road Traffic Fine Management Process (177 MB, 561,470 events) as a stress test. Both datasets are published benchmarks widely used in the process mining community.

The result was not “`pictl` failed.” The result was: $\Delta = 41$ algorithms, structured as follows:

- 0 algorithms at Tier 0 (press release target)
- 9 algorithms at Tier 1 (correct output, performance gap only)
- 24 algorithms at Tier 2 (architectural defect: `RefCell<Mutex>` in WASM)
- 8 algorithms at Tier 3 (no WASM export: press release, not implementation)

Benchmark Results from Real Data

The audit ran against two real event logs: Sepsis Cases (5 MB, fast feedback cycle) and Road Traffic Fine Management Process (177 MB, 561K events, stress test). Measured performance:

Algorithm	Status	Sepsis (5MB)	Road Traffic (561K)
DFG	Tier 1	1.19 ms	pass
Heuristic Miner	Tier 1	45 ms	pass
A*	Tier 1	120 ms	pass
Genetic Algo.	Tier 2	crash	crash
ACO	Tier 2	crash	crash
PSO	Tier 2	crash	crash
yawl_export	Tier 3	not exported	—
ILP	Tier 3	not exported	—

The DFG implementation achieved 118.9 M events/second throughput on modern hardware, demonstrating that the press release targets (5s on 500K events) are achievable for the fast-path algorithms. The Tier 2 crash pattern (24 algorithms) was identical across datasets, confirming an architectural root cause rather than data-dependent edge cases.

This is a gap measurement, not a verdict. The gap is large. Some of it (Tier 1) is closable with performance work. Some of it (Tier 2) requires architectural rewrite. Some of it (Tier 3) was always aspirational.

5.3. The Plan Revision: A Closer-Horizon Press Release

Given the gap measurement, the plan agent faced a decision:

1. **Close the existing gap:** Rewrite 24+ algorithms in WASM-safe Rust, add behavioural oracles, rebuild the conformance pipeline. Time estimate: months. Risk: high (architectural root cause may have further ramifications).
2. **Revise the press release:** Write a new press release for a closer horizon—a system where the gap is small enough to close in days, and where the oracle problem is solved by mathematical theorems rather than probabilistic quality metrics.

The second option was chosen. The new press release is **bcinr**: 12 algorithm families, ≈ 45 functions, all operating on primitive types, all verifiable by mathematical theorems expressible as unit tests.

This is not abandonment. It is **Working Backwards from a closer milestone**. The **bcinr** press release is achievable in days. When it is delivered, a new press release for the next milestone can be written. The frontier advances.

5.4. The bcinr Press Release Written in Code

The **bcinr** ontology (**bcinr.ttl**) is the new press release, written in RDF:

```

1 <http://bcinr.rs/instance/family/mask> a bcinr:AlgorithmFamily ;
2   bcinr:id "mask" ;
3   bcinr:label "Mask Calculus" ;
4   bcinr:description "Bitwise mask selection: (mask & a) | (~mask & b)" .
5
6 <http://bcinr.rs/instance/symbol/mask_select_u32> a bcinr:ApiSymbol ;
7   bcinr:ofFamily <.../family/mask> ;
8   bcinr:name "select_u32" .

```

This says: *we are building a library that includes a Mask Calculus family with a `select_u32` function*. The gap between this press release and reality is precisely measurable by asking: does `cargo test` pass for `mask::select_u32`?

The oracle is mathematical:

$$\text{select_u32}(0xFFFF_FFFF, a, b) = a \quad \forall a, b \in \mathbb{Z}_{2^{32}}$$

This is a Rank-1 oracle—a mathematical theorem that is true before any implementation exists. The gap measurement is not probabilistic. The press release is either delivered or it is not, and the test tells you which.

6. Implications for Agent-Driven Development

6.1. The Plan File Must Be the Press Release, Not the Task List

A plan file that lists tasks is a sprint backlog. It has all the failure modes of sequential task cycles: it measures completion, not gap closure; it severs tasks from the press release; it cannot represent the state “this task should not be completed.”

A plan file that is the press release is different:

- It describes desired outcomes, not implementation steps.
- It is rewritten (not amended) when the product vision changes.
- It contains the gap measurement as a first-class element: what is the distance between this vision and current reality?
- It expires. When the press release is delivered, a new one is written for the next horizon.

The plan mode gate in Claude Code—requiring human approval before write agents begin—enforces this distinction. The human reads the press release (plan file) and approves the gap-closure strategy (write phase) before any state changes. They cannot accidentally approve a write phase that is implementing toward a stale press release, because the press release is what they are approving.

6.2. Explore Agents Run in Parallel Because Gaps Are Structural

A single explore agent reading one file produces a local fact. A fleet of explore agents reading the whole system simultaneously produces a structural fact—a pattern visible only when all the data arrives together.

This is why Claude Code launches multiple explore agents in one message. It is not a performance optimisation. It is an epistemological requirement: the gap between the press release and reality is a global property of the system, not a local property of any file.

The `pictl` architectural finding—`RefCell<Mutex>` incompatible with WASM—was a structural fact. It required seeing all 41 algorithm results simultaneously. A sequential audit would have seen it as 41 local facts, each ambiguous in isolation.

6.3. False Completion Is Press Release Revision, Not Failure

When the `pictl` audit concluded that the Tier 2 gap was not worth closing, this was not a declaration of failure. It was a **press release revision**: the product vision moved from “41-algorithm process mining platform” to “12-family branchless algorithm library with mathematical oracles.”

The old press release was not wrong. It was superseded. The gap measurement made it possible to compute:

$$\mathbb{E}[\text{value} \mid \text{close Tier 2 gap}] < \mathbb{E}[\text{value} \mid \text{revise press release, closer horizon}]$$

This computation is only possible if the gap has been measured. Sequential task cycles never measure the gap. They close tickets. They cannot make this computation because they have no representation of the distance between the current state and the press release.

7. Conclusions

The Amazon Working Backwards press release is not a lie. It is a design instrument: a future state written in present tense to pull a team toward it. The press release is correct at the time of writing because it describes the intended system, not the current system. The gap between them is the work.

The `pictl` registry was a press release written in code. The 41 algorithms were design commitments, not implementation claims. The error was not making the commitments. The error was failing to measure the gap systematically and failing to update the press release when the gap proved too large to close on the current horizon.

The Explore/Plan/Write agent architecture is the computational implementation of Working Backwards:

- The **plan file** is the press release: desired state, written before implementation, revised when the product changes.
- **Explore agents** measure the gap: parallel, structural, architectural facts, not sequential local facts.
- **Write agents** close the gap: specific, bounded, directed by the plan, not self-directed.

The sequential task cycle fails frontier work because it has no press release. Tickets are islands. Velocity measures ticket closure, not gap closure. The team moves fast toward a destination it has stopped looking at.

Dream big. Write the press release first. Measure the gap. Close what you can. Revise the horizon. Repeat.

The `pictl` registry and the `bcinr` ontology are both press releases. Neither is a lie. Both are the correct starting point for the work they represent. The difference is that `bcinr` has oracles—and oracles are what turn press releases into delivery schedules.

Acknowledgements

The author thanks Jeff Bezos and the Amazon Working Backwards process for the press-release-first discipline; Wil van der Aalst for the process mining doctrine that served as the primary oracle throughout the `pictl` arc; and the Anthropic Claude Code team for the Explore/Plan/Write agent architecture that made the gap-measurement methodology concrete and executable.

References

- [1] C. Watt and I. McAllister. *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. St. Martin's Press, 2021.
- [2] W. M. P. van der Aalst. *Process Mining: Data Science in Action*, 2nd ed. Springer, 2016.
- [3] P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2-3):235–256, 2002.

-
- [4] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
 - [5] G. Kim, P. Humble, J. Willis, and J. Debois. *The DevOps Handbook*. IT Revolution Press, 2016.